

SaludBoyacá

Gestión de Citas Médicas

Preguntas de Reflexión — Taller i18n + Docker + OTP

Aprendiz:	Steven Araque
Programa:	Tecnólogo en Análisis y Desarrollo de Software — ADSO
Centro:	CIMM — Centro Industrial de Mantenimiento y Manufactura
Regional:	Boyacá
Ficha:	3171062
Competencia:	225501003 — Desarrollar e integrar componentes y servicios

11.1 Código de Referencia y Arquitectura

Pregunta 1: LoginServlet — similitudes y diferencias entre el sistema de vacunación y SaludBoyacá

Al comparar el **LoginServlet** del sistema de vacunación con el de SaludBoyacá, aproximadamente el **60 % del código es idéntico o muy similar**: la estructura `doGet/doPost`, la lectura de parámetros con `request.getParameter()`, el manejo de sesión con `HttpSession`, el uso de `ResourceBundle` para textos i18n y la redirección con `sendRedirect()` o `forward()`. Lo que cambió completamente fue el bloque de autenticación: en vacunación se valida el CAPTCHA gráfico y luego se redirige directamente al dashboard; en SaludBoyacá se eliminó todo el bloque del CAPTCHA y en su lugar se genera un código OTP con `OTPService.generarOTP()`, se guarda en sesión y en la tabla `otp_tokens`, y se redirige a `/otp`. Esto demuestra el valor de estudiar código bien estructurado: no se parte de cero, se entiende el patrón y solo se modifica lo que cambia por dominio o requerimiento, reduciendo errores y tiempo de desarrollo.

Pregunta 2: El patrón switch/acción en PacienteServlet — por qué es repetible y qué ocurriría sin él

El patrón **switch/acción** (leer el parámetro `accion` y ejecutar el método correspondiente: listar, nuevo, guardar, editar, eliminar) es repetible porque separa la decisión de control de la lógica de negocio. Cada Servlet actúa como un "mini-controlador" con una única URL y múltiples operaciones. En SaludBoyacá, **CitaServlet** y **PacienteServlet** replican exactamente este patrón: el Servlet no contiene SQL, delega en el DAO y reenvía a la vista JSP correcta. Si cada Servlet tuviera una estructura completamente diferente, el mantenimiento sería caótico: un nuevo integrante tendría que aprender el estilo de cada clase por separado, los errores serían más difíciles de rastrear y agregar una nueva acción implicaría entender lógica inconsistente. La uniformidad hace el código predecible, testeable y escalable.

Pregunta 3: Comparación del esquema BD de vacunación (5 tablas) vs SaludBoyacá (7 tablas)

El esquema de vacunación tiene 5 tablas: `usuarios`, `pacientes`, `vacunas`, `registros_vacunacion`, `log_cambios`. De ese modelo se aprendió el uso de FK bien definidas, el campo ENUM para roles, la tabla de log de auditoría

y la separación entre la entidad principal (registro) y sus catálogos (vacunas). SaludBoyacá requiere 7 tablas porque el dominio es más complejo: se agregaron **especialidades** (catálogo independiente), **horarios** (disponibilidad por médico y día) y **otp_tokens** (seguridad de autenticación). La tabla *log_accesos* reemplaza a *log_cambios* con un enfoque centrado en autenticación. El campo *lang_preferido* en usuarios no existe en vacunación y responde al requerimiento i18n multiusuario de SaludBoyacá. Cada decisión de diseño tuvo justificación funcional, no fue arbitraria.

11.2 Internacionalización (i18n) y OTP

Pregunta 4: LocaleFilter — por qué es reutilizable y cómo agregar un nuevo idioma

LocaleFilter es reutilizable porque no contiene ninguna lógica de negocio del dominio: solo detecta el parámetro *?lang=XX* en cada petición, valida que sea un idioma permitido, lo guarda como atributo de sesión (*lang* y *locale*) y llama a *chain.doFilter()*. Al estar anotado con *@WebFilter("//*")* intercepta toda la aplicación sin que ningún Servlet ni JSP tenga que llamarlo explícitamente. Esa independencia total del dominio lo hace copiable casi sin cambios a cualquier proyecto Java EE. Para agregar portugués (**pt**) bastaría con: (1) agregar "pt" al array de idiomas válidos en el filtro, (2) crear el archivo *messages_pt.properties* con todas las claves traducidas, y (3) agregar el botón de la bandera brasileña en el selector de idioma de *header.jsp*. No se toca ningún Servlet ni DAO.

Pregunta 5: OTP reemplaza al CAPTCHA en el login — por qué no se usan juntos y dónde persiste el CAPTCHA

El CAPTCHA protege contra bots automatizados que prueban credenciales en masa. El OTP cumple esa misma función (un bot no tiene acceso al correo del usuario real) pero además **verifica la identidad**: demuestra que quien inicia sesión controla la bandeja de correo registrada en la base de datos. Usarlos juntos sería redundante y degradaría la experiencia: el usuario tendría que superar dos barreras para el mismo objetivo. El OTP es superior porque protege más y, al mismo tiempo, es el segundo factor de autenticación. El CAPTCHA se conserva en el módulo de **consulta pública** (*/consulta-cita*) precisamente porque allí **no existe cuenta de usuario**: el ciudadano solo ingresa su número de documento y no tiene un correo registrado en el sistema al cual enviar un OTP. Sin cuenta no hay correo; sin correo no hay OTP posible. El CAPTCHA visual es, en ese contexto, el único mecanismo anti-bot viable.

Pregunta 6: SecureRandom vs Random — por qué guardar el OTP en BD es más seguro que solo en sesión

SecureRandom usa fuentes de entropía del sistema operativo (ruido de hardware, eventos del kernel) para generar números impredecibles criptográficamente. La clase **Random** usa un algoritmo lineal congruencial que, conociendo la semilla o suficientes salidas anteriores, puede predecirse. Para un código de seguridad de 6 dígitos, la predictibilidad de *Random* es un riesgo real. Guardar el OTP solo en sesión HTTP tiene varias debilidades: si el servidor se reinicia, la sesión se pierde; si el usuario abre otra pestaña o dispositivo, las sesiones divergen; y no hay forma de auditar intentos fallidos. Guardarlo en la tabla **otp_tokens** resuelve todo: persiste ante reinicios, permite marcar el token como *usado* para evitar replay attacks (reutilización del mismo código), registra la fecha de expiración (*expira_en*) con precisión de DATETIME y permite auditoría completa. La BD es la fuente de verdad; la sesión es solo un caché temporal.

11.3 Docker, Despliegue y Decisiones de Producción

Pregunta 7: Multi-Stage Build — por qué importa el tamaño de la imagen en hosting con recursos limitados

Sin Multi-Stage Build, usando una sola etapa *maven:3.8.6-openjdk-14*, la imagen resultante pesaría entre **600 y 800 MB** porque incluye el JDK completo, el binario de Maven, las dependencias descargadas en caché y todos los archivos temporales de compilación. Con Multi-Stage Build, la etapa de build compila el WAR y la etapa de ejecución solo toma el WAR compilado y lo copia a una imagen limpia de *tomcat:9.0-jdk14-openjdk*, resultando en **~250-300 MB**. Ese ahorro es crítico en **Back4App** (256 MB de RAM total): una imagen pesada haría que Tomcat no pudiera arrancar por falta de memoria, de ahí que también se requiera configurar *JAVA_TOOL_OPTIONS=-Xms64m -Xmx200m*. En **Koyeb** con 0.1 vCPU, el tiempo de pull de la imagen desde Docker Hub impacta el tiempo de arranque; una imagen más pequeña se descarga más rápido y el servicio queda disponible antes.

Pregunta 8: Proceso de despliegue en 2 hosting — pasos, errores encontrados y cómo depurarlos

Render: Se creó el servicio desde *render.com* seleccionando "Deploy an existing image from a registry", se ingresó la imagen *stevenalejandro/saludboyaca:v1* de Docker Hub, se configuraron las variables de entorno *DB_URL*, *DB_USER*, *DB_PASS*, *EMAIL_PASS* y se seleccionó el plan Free. El despliegue tardó aproximadamente 4 minutos. El primer error fue **Cannot connect to database**: la variable *DB_URL* tenía el host de Railway mal copiado (faltaba el puerto al final). Se corrigió revisando la pestaña Variables de Railway y copiando la cadena JDBC completa. El segundo error fue que la aplicación cargaba en blanco: Tomcat había desplegado el WAR pero el contexto raíz no resolvía porque el WAR no se llamaba *ROOT.war*. Se corrigió en el Dockerfile cambiando el nombre de copia a *ROOT.war*. Los logs se consultan en la pestaña "Logs" del dashboard de Render en tiempo real. **Koyeb:** Se creó el servicio con la misma imagen y se configuró el puerto 8080. El error más frecuente fue **Application failed to start** porque Koyeb asigna un puerto dinámico via la variable *PORT*; Tomcat arranca siempre en 8080 y eso generaba un health check fallido. Se resolvió configurando explícitamente el Container Port en 8080 dentro del panel de Koyeb. Otro error fue **AuthException: 535 Authentication failed** al enviar el OTP, causado porque *EMAIL_PASS* fue pegado con comillas dobles extra en el campo de variable de entorno. Se corrigió eliminando las comillas del valor en el panel. Los logs se ven en tiempo real desde la sección "Service Logs" del dashboard de Koyeb.

Pregunta 9: Recomendación de hosting para producción real con 50 usuarios simultáneos

Evaluando los 5 hosting del taller para un entorno de producción real con **50 usuarios simultáneos** en el Centro de Salud de Paipa, se descartan primero las opciones insuficientes: **Render** queda eliminado por el sleep de 15 minutos sin actividad, inaceptable en un sistema de salud donde cualquier funcionario puede necesitar acceso urgente. **Back4App** queda eliminado por el límite de 256 MB de RAM, insuficiente para Tomcat con carga real. **Google Cloud Run** escala a cero cuando no hay tráfico, lo que genera cold starts de varios segundos en la primera petición tras un periodo de inactividad; esto es aceptable para aplicaciones de alto volumen, pero en un centro de salud municipal con tráfico irregular puede generar demoras inesperadas. La primera recomendación es **Oracle Cloud Always Free**: ofrece hasta 4 OCPU y 24 GB de RAM en VMs ARM Ampere A1, recursos suficientes para Tomcat con múltiples hilos concurrentes, una instancia MySQL local y monitoreo. Siempre activo, sin cold starts y sin límite de peticiones. Su única desventaja es la complejidad inicial de configuración (SSH, Docker manual, apertura de puertos), pero para un entorno permanente esa inversión se justifica. La **segunda opción** es **Koyeb**: siempre activo sin cold starts, 512 MB de RAM, SSL automático y despliegue desde Docker Hub en menos de 2 minutos, sin la complejidad operativa de una VM. Para 50 usuarios con carga moderada (citas, consultas, generación de PDF) Koyeb es suficiente y más fácil de mantener.

Repositorio del Proyecto en GitHub

El código fuente completo de SaludBoyacá está disponible públicamente en:

<https://github.com/stevenaraque/saludboyaca.git>

Incluye: código fuente Java · JSP · script SQL · Dockerfile · README.md con instrucciones de despliegue, credenciales de prueba y capturas de pantalla de las 6 pantallas principales del sistema.

SENA · Centro Industrial de Mantenimiento y Manufactura (CIMM) · Regional Boyacá · Tecnólogo ADSO · 2026